

情報学群実験第3 ベアメタルプログラミング

細川 夏風

2026 年 6 月 14 日

1 目的

プログラミングは計算機の中で行われることは、言うまでもない。しかし、計算機だけでは単純にプログラムを行うことは非常に難解である。プログラムはそれを実行する環境などの土台が必要である。今実験ではプログラムを実行する環境、またそのために計算機自体を制御する方法を C 言語で簡易的なストップウォッチのプログラムを作成する事によって明らかにする。

2 方法

前提として、付録に定数やデバイスの制御に用いるプログラムとベアメタル環境において C 言語を動作させるためのアセンブリプログラムを掲載している。

また、班員と共同制作したストップウォッチのプログラム本体も付録に掲載している。

3 実験環境

本実験におけるハードウェア環境およびソフトウェアツールの詳細を以下に示す。

3.1 ハードウェア仕様およびレジスタの配置

- **Raspberry Pi および SoC アーキテクチャ**

本実験では、Broadcom 社の SoC である BCM2835 の仕様をベースとする。ただし、実際のハードウェア（Raspberry Pi 3 など）に搭載されている SoC（BCM2837 等）のメモリマップに準拠して制御を行う。公式ドキュメント上のベースアドレスは 0x20000000 であるが、本実験の環境では 0x3F000000 に配置されている。

- **GPIO レジスタ (0x3F200000)**

ベースアドレスからオフセット 0x00200000 に配置されている。本実験では、GPFSEL0 ~ 2（機能選択）、GPSET0（出力 High）、GPCLR0（出力 Low）、GPLEV0（入力読み取り）などのレジ

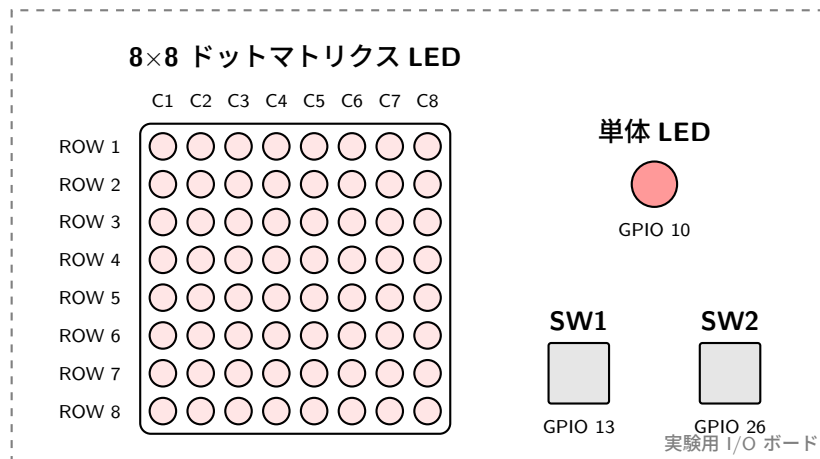


図 1 実験用 I/O ボード上の LED およびスイッチの論理配置図

スタを介して外部装置の制御を行う。

- システムタイマー (0x3F003000)

ベースアドレスからオフセット 0x00003000 に配置されているフリーランニングカウンタ (TMCL0) を使用する。動作周波数は 1 MHz であり、カウンタ値が 1 増加することで $1\mu\text{s}$ の経過を表す。

4 開発・ソフトウェア環境

本実験におけるプログラムの開発、コンパイル、およびビルドには、以下のクロスツールチェーンを使用する。

- クロスコンパイル環境 (ARM ツールチェーン)

開発用の PC 上で、ターゲットとなる ARM アーキテクチャ (Raspberry Pi) 用の実行ファイルを生成するために、以下の arm-none-eabi- 系のツールを使用する。

- arm-none-eabi-gcc

C 言語で記述されたソースコードを、ARM 用のオブジェクトコードに翻訳する C コンパイラ。

- arm-none-eabi-as

アセンブリ言語で記述されたコードを、ARM 用のオブジェクトコードに翻訳するアセンブラ。

- arm-none-eabi-ld

コンパイラやアセンブラが生成した複数のオブジェクトファイルを結合 (リンク) し、一つの実行ファイル (ELF 形式など) を生成するリンカ。

- arm-none-eabi-objcopy

生成された実行ファイルを、Raspberry Pi 上で直接実行可能な形式 (ベタバイナリ) の

.img ファイルなど) に変換するためのファイル形式変換ツール.

5 実験用 I/O ボードの仕様

本実験で使用する I/O ボードに接続された各装置の回路仕様, および制御方法を以下に示す.

5.1 ドットマトリクス LED の制御仕様

8×8 ドットマトリクス LED の表示は, 行を高速に切り替える方式 (ダイナミック点灯方式) で制御を行う.

5.1.1 表示制御のアルゴリズム

マトリクスの行 (ROW1 ~ ROW8) を順に 1 行だけ表示させる操作を高速に繰り返す. 具体的な手順は以下の通りである.

1. まず, すべての行 (ROW) に 1 を出力しておく.
2. 行 r を表示させるには, 点灯させる列 (COL) には 1 を, 消灯させる列には 0 を出力する.
3. そのうえで, 対象の行 ROW r に 0 を出力する.

5.2 スイッチの回路仕様

I/O ボード上の押しボタンスイッチの回路構成は以下の通りである.

- 10 k Ω の抵抗でプルダウンされている.
- 誤電流防止およびポート保護のため, 1 k Ω の保護抵抗が直列に挿入されている.

5.3 実験用 I/O ボードのピン割り当て

Raspberry Pi の GPIO ピンに接続されている実験用ボード上の装置 (LED, スイッチ, ドットマトリクス LED) の対応関係を表 1 に示す.

5.4 プログラムのビルドおよびデプロイ手順

ソースコード (アセンブリおよび C 言語) から実行可能なバイナリイメージを生成し, Raspberry Pi へ書き込むまでの具体的なコマンド実行手順を以下に示す.

```
$ arm-none-eabi-as init.s -o init.o
$ arm-none-eabi-gcc -c $x.c -o $x.o
$ arm-none-eabi-ld -m armelf init.o $x.o -o $x.elf
$ arm-none-eabi-objcopy $x.elf -O binary $x.img
$ cp $x.img /boot ボリュームのパス名/kernel7.img
```

表1 実験用 I/O ボードの GPIO ピン割り当て

装置名	GPIO 番号	マトリクス列 (COL)	GPIO 番号	マトリクス行 (ROW)	GPIO 番号
単体 LED	10	COL1	27	ROW1	14
SW1	13	COL2	8	ROW2	15
SW2	26	COL3	25	ROW3	21
		COL4	23	ROW4	18
		COL5	24	ROW5	12
		COL6	22	ROW6	20
		COL7	17	ROW7	7
		COL8	4	ROW8	16

各コマンドの具体的な役割と処理内容は以下の通りである。

- **アセンブル (arm-none-eabi-as)**

スタートアップルーチンが記述されたアセンブリファイル (`init.s`) をアセンブルし、オブジェクトファイル (`init.o`) を生成する。

- **コンパイル (arm-none-eabi-gcc)**

C 言語のソースコード (`light.c`) をコンパイルし、オブジェクトファイル (`light.o`) を生成する。ここで `-c` オプションを指定することで、リンク処理を行わずにコンパイルのみ (オブジェクトファイルの生成) で処理を停止させている。

- **リンク (arm-none-eabi-ld)**

生成された2つのオブジェクトファイル (`init.o` と `light.o`) を結合し、一つの ELF 形式の実行ファイル (`light.elf`) を生成する。`-m armelf` は、ARM アーキテクチャ向けの ELF 形式であることを指定するオプションである。

- **フォーマット変換 (arm-none-eabi-objcopy)**

ELF 形式の実行ファイルから OS 向けの不要なヘッダ情報等を除去し、Raspberry Pi (ベアメタル環境) 上で直接実行可能なバイナリ形式 (`light.img`) へ変換する (`-O binary` オプション)。

- **デプロイ (cp)**

完成したバイナリイメージを、SD カードのブートパーティションへコピーする。Raspberry Pi 2 や 3 の GPU ファームウェアは、起動時に `kernel7.img` という名前のカーネルイメージを自動的に読み込んで実行を開始する仕様となっているため、このファイル名に変更して保存している。

6 実行結果

演習 4.3 の仕様にに基づき実装したストップウォッチプログラム (stopwatch.c) を実験用 I/O ボード上で実行し、以下の正常動作を確認した (または、以下の動作が期待される)。

- 初期状態

プログラムの実行開始直後、ドットマトリクス LED には「00」が表示される。また、ストップウォッチのカウント状態には一切依存せず、単体 LED が常時 1 秒周期 (0.5 秒点灯, 0.5 秒消灯) で点滅を開始し、継続する。

- 動作開始 (スイッチ 1 押下時)

スイッチ 1 (SW1) を押下すると、ドットマトリクス LED の数値が「00」から開始し、0.1 秒 (100 ミリ秒) 経過するごとに「01」、「02」とカウントアップしていく。数値が「99」に達した次の 0.1 秒後には「00」に戻り、ループしてカウントアップを継続する。

- 動作停止 (スイッチ 2 押下時)

カウントアップ中にスイッチ 2 (SW2) を押下すると、その瞬間の数値を保持したままドットマトリクス LED のカウント動作が停止する。なお、カウント停止中であっても、単体 LED の 1 秒周期の点滅処理は影響を受けずに独立して継続する。

- 動作再開 (再びスイッチ 1 押下時)

停止状態から再度スイッチ 1 (SW1) を押下すると、停止時に保持されていた数値からストップウォッチのカウントアップが直ちに再開される。

7 考察

本実験の演習 4.3 における実装結果に基づき、OS が存在しないベアメタル環境における多重処理 (マルチタスク) の実現手法、およびメモリ初期化がシステムの安定動作に与える影響について考察する。

7.1 ベアメタル環境における並行処理 (マルチタスク) の手法

本演習では、タスクスケジューラやスレッド管理を行う OS が存在しない環境において、以下の 4 つの処理を並行して実行する必要があった。

1. 8×8 ドットマトリクス LED の高速な行切り替え
2. 1 秒周期の単体 LED 点滅処理
3. 0.1 秒周期のストップウォッチのカウントアップ処理
4. スイッチ 1 (SW1) およびスイッチ 2 (SW2) の入力状態の監視

これらを同時に破綻なく実行するためには、実行を完全に停止させてしまう単一の長い for 文に

おける遅延ループを一切排除し、ハードウェアシステムタイマー (TMCL0) の値を監視する時分割処理および状態遷移の設計が不可欠である。

具体的には、メインループ内で常にドットマトリクス LED の各行を 1 行ずつ高速にスキャンしつつ、単体 LED やカウントアップの処理に対しては前回の実行時刻からの経過時間をタイマーで都度判定する。設定された時間 (0.5 秒や 0.1 秒) に達した瞬間のみ対応する状態の更新を行い、それ以外の時間は直ちにスキャン処理やスイッチ監視へ制御を戻す。このように、処理を細切れにして CPU を巡回させることで、単一コアであってもすべての処理が擬似的に並行して動くマルチタスク動作が可能となる。

7.2 フレームバッファ初期化の重要性と未初期化時の挙動

ドットマトリクス LED の点灯パターンを保持する配列 (フレームバッファ) の初期化は、プログラム起動時の正常な挙動を保証するために極めて重要である。

C 言語において、関数外で宣言された初期値のないグローバル変数は BSS セクションに配置され、関数内で宣言されたローカル変数はスタック領域に配置される。一般的な OS 環境では、プログラム実行時に OS がこれらの領域を自動的に 0 で初期化する。しかし、本実験のようなベアメタル環境では、スタートアップルーチンで明示的にメモリ領域をクリアする処理を入れていない場合、または C 言語のプログラムの冒頭で配列に初期値を代入しない場合、割り当てられたメモリの内容は起動直後のランダムなゴミデータとなる。

もしフレームバッファの初期化を行わずにダイナミック点灯処理を開始した場合、配列内の不定値がそのまま「点灯」の判定条件として評価されてしまう。その結果、ストップウォッチが動作を始める前の初期状態において、意図しない位置の LED が不規則に点灯したり、画面全体にノイズが表示されたりするバグが発生する。このことから、組み込みシステム開発においては、使用するすべてのメモリを明示的に初期化するコードを記述することが、システムの予測可能性と信頼性を担保する上で必須の作法であると言える。

A ヘッダファイル const.h

本付録では、GPIO およびタイマー操作用に定義された各種ベースアドレスとポート番号の設定ファイル const.h を示します。

Listing 1 const.h のソースコード

```
1 // GPIOを操作するための番地
2 #define GPIO_BASE      (volatile int *)0x3f200000
3 #define GPFSEL0        (GPIO_BASE + 0) // GPIO #0～9の機能選択
4 #define GPFSEL1        (GPIO_BASE + 1) // GPIO #10～19の機能選択
5 #define GPFSEL2        (GPIO_BASE + 2) // GPIO #20～29の機能選択
6 #define GPSET0         (GPIO_BASE + 7) // 出力値を1にする
```

```

7  #define GPCLR0          (GPIO_BASE + 10) // 出力値を0にする
8  #define GPLEV0          (GPIO_BASE + 13) // 入力値を読み出す
9
10 // GPFSELxに設定する値
11 // 前回分
12 // #define GPFSEL_VEC1      1    // GPFSEL1用（#10のみ出力）
13
14 // 今回分
15 #define GPFSEL_VEC0      0x01201000 // GPFSEL0用（#4, #7, #8 が出力）
16 #define GPFSEL_VEC1      0x01249041 // GPFSEL1用（#10, #12, #14～18 が
    出力）
17 #define GPFSEL_VEC2      0x00209249 // GPFSEL2用（#20～25, #27 が出力）
18
19 // 各装置が接続されているGPIO番号
20 #define LED_PORT          10 // LED
21 #define SW1_PORT          13 //PORT OF SWITCH1
22 #define SW2_PORT          26 //PORT OF SWITCH2
23
24 #define COL1_PORT         27
25 #define COL2_PORT         8
26 #define COL3_PORT         25
27 #define COL4_PORT         23
28 #define COL5_PORT         24
29 #define COL6_PORT         22
30 #define COL7_PORT         17
31 #define COL8_PORT         4
32
33 #define ROW1_PORT         14
34 #define ROW2_PORT         15
35 #define ROW3_PORT         21
36 #define ROW4_PORT         18
37 #define ROW5_PORT         12
38 #define ROW6_PORT         20
39 #define ROW7_PORT         7
40 #define ROW8_PORT         16
41

```

```

42  #define TIMER_BASE (volatile unsigned int *)0x3f003000
43
44  #define TMCL0 (TIMER_BASE + 1)
45  #define TIMER_HZ    (1000 * 1000)    /* タイマの周波数 (= 1秒を表すカウンタ値) */

```

B ストップウォッチプログラム本体

班員と共同作成したストップウォッチのプログラムの本体をここに掲載する.

Listing 2 stopwatch.c のソースコード

```

1  // スイッチが押されているかどうか if (sw & (1 << SW1_PORT))
2
3
4  #include "const.h"    // 定数の定義を読み込む
5
6  // Frame バッファを使うプログラム
7
8  char frame_buffer[10][8] = {
9      { 0x0, 0xe, 0xa, 0xa, 0xa, 0xe, 0x0, 0x0 }, // 0
10     { 0x0, 0x4, 0xc, 0x4, 0x4, 0xe, 0x0, 0x0 }, // 1
11     { 0x0, 0xe, 0x2, 0xe, 0x8, 0xe, 0x0, 0x0 }, // 2
12     { 0x0, 0xe, 0x2, 0xe, 0x2, 0xe, 0x0, 0x0 }, // 3
13     { 0x0, 0xa, 0xa, 0xe, 0x2, 0x2, 0x0, 0x0 }, // 4
14     { 0x0, 0xe, 0x8, 0xe, 0x2, 0xe, 0x0, 0x0 }, // 5
15     { 0x0, 0xe, 0x8, 0xe, 0xa, 0xe, 0x0, 0x0 }, // 6
16     { 0x0, 0xe, 0x2, 0x2, 0x2, 0x2, 0x0, 0x0 }, // 7
17     { 0x0, 0xe, 0xa, 0xe, 0xa, 0xe, 0x0, 0x0 }, // 8
18     { 0x0, 0xe, 0xa, 0xe, 0x2, 0xe, 0x0, 0x0 } // 9
19 };
20
21 int main()
22 {
23     // GPIOの機能設定
24     *GPFSEL0 = GPFSEL_VEC0; // #0～#9の設定
25     *GPFSEL1 = GPFSEL_VEC1; // #10～#19の設定
26     *GPFSEL2 = GPFSEL_VEC2; // #20～#29の設定

```

```

27
28 // GPLEV0の値（GPIO #0～#31それぞれが読み取った値）をswに代入
29 int sw;
30 int move; //ストップウォッチを動作させるか停止させるか
31 move = 0; //最初は停止させている
32
33 char colx_port[] = {
34     COL1_PORT,
35     COL2_PORT,
36     COL3_PORT,
37     COL4_PORT,
38     COL5_PORT,
39     COL6_PORT,
40     COL7_PORT,
41     COL8_PORT
42 };
43
44 char rowx_port[] = {
45     ROW1_PORT,
46     ROW2_PORT,
47     ROW3_PORT,
48     ROW4_PORT,
49     ROW5_PORT,
50     ROW6_PORT,
51     ROW7_PORT,
52     ROW8_PORT
53 };
54
55 int row, i, x, k, j, x_up, x_down;
56
57 // タスクの時間設定
58 unsigned int time, target, target_display_row, number_timer,
59     targetLED_on, targetLED_off;
60
61 // 時間表示は1秒ごと
number_timer = *TMCL0 + TIMER_HZ;

```

```

62 // 画面表示は0.001秒ごと
63 target_display_row = *TMCL0 + 1000;
64 // LEDのON・OFFは0.5秒周期
65 targetLED_on = *TMCL0 + TIMER_HZ / 2;
66 targetLED_off = targetLED_on + TIMER_HZ / 2;
67
68 // x は表示する数
69 x = 0;
70 row = 0;
71
72 // 表示する数を格納したもの
73 char new_frame[8];
74
75 // 最初の1秒間（1秒タスクが動くまで）に表示するための「初期データ
    (0)」を作っておく
76 x_up = x / 10;
77 x_down = x % 10;
78 for (k = 0; k < 8; k++) {
79     new_frame[k] = (frame_buffer[x_up][k] << 4) + frame_buffer[x_down
        ][k];
80 }
81 while(1){
82
83     // GPLEV0の値（GPIO #0～#31それぞれが読み取った値）をswに代入 現
        在の状態を読み取る
84     sw = *GPLEV0;
85     // 現在時刻をtimeに代入
86     time = *TMCL0;
87
88     //スイッチ1を押すと動作を開始，再開する
89     if (sw & (1 << SW1_PORT)) {
90         move = 1;
91     }
92     //スイッチ2を押すと動作を停止する
93     if (sw & (1 << SW2_PORT)) {
94         move = 0;

```

```

95     }
96     //以降のタスクでmove && をとる
97
98     // 消灯と点灯のタスク
99     if (time >= target_display_row) {
100         // 前の行の消灯
101         *GPSET0 = 1 << rowx_port[row];
102         row = (row + 1) % 8;
103
104         int column;
105         // 表示配列の1数字ずつ読む
106         for (column = 0; column < 8; column++) {
107             // 1のときは点灯の配置0のときは消灯の配置
108             if ((new_frame[row] >> (7 - column)) & 1) {
109                 *GPSET0 = 1 << colx_port[column];
110             } else {
111                 *GPCLR0 = 1 << colx_port[column];
112             }
113         }
114
115         // 次の行の点灯
116         *GPCLR0 = 1 << rowx_port[row];
117         target_display_row += 1000;
118     }
119
120     // バッファにしたがって1~8出力を変更
121     // バッファ変更のタスク
122     if (move && (time >= number_timer)) {
123         x = (x + 1) % 100;
124         // 10進数の1と10の桁
125         x_up = x / 10;
126         x_down = x % 10;
127         // 初期フレームバッファの4bitシフトしたもの(10の桁)と通常(1の桁)
128         // を足した値を生成する
129         for (k = 0; k < 8; k++) new_frame[k] = (frame_buffer[x_up][k]
130             << 4) + frame_buffer[x_down][k];

```

```

129         // 次の時間は0.1秒先
130         number_timer += 100000;
131     }
132
133     // 0.5秒点灯
134     if (time >= targetLED_on) {
135         *GPSET0 = 1 << LED_PORT; //LEDを点灯する;
136         targetLED_on += TIMER_HZ; // 1秒足す
137     }
138     if (time >= targetLED_off) {
139         *GPCLR0 = 1 << LED_PORT; //LEDを消灯する
140         targetLED_off += TIMER_HZ; // 1秒足す
141     }
142 }
143 return 0; // init.sに戻る
144 }

```

C 最小のスタートアップルーチン

OSが存在しない環境において、C言語のmain関数を呼び出すための最小のスタートアップルーチンとして以下のアセンブリプログラムを掲載する。

Listing 3 最小のスタートアップルーチン

```

1  @ 最小限のスタートアップルーチン
2  .section .init
3  .global _start
4  _start:
5      ldr    sp, =0x8000    @ スタックポインタ初期化
6      bl     main          @ main関数呼び出し
7      b      .             @ その場で無限ループ（停止）

```